

AI Task Processing Platform - Architecture Document

Prepared for technical assessment submission

July 2026

1. Overall System Architecture

The platform follows a standard MERN + async-worker topology, chosen so the request path (auth, task creation, status reads) stays fast and stateless, while the actual “AI processing” work happens off the request/response cycle entirely.

Request flow:

1. The **React/Vite frontend** calls the **Node.js/Express API** over HTTPS for auth (register/login, JWT-based) and task CRUD.
2. On `POST /api/tasks`, the API writes a `Task` document to **MongoDB** with `status: pending`, then pushes the task’s ID onto a **Redis** list (`ai_task_queue`) with `LPUSH`.
3. One or more **Python worker** replicas block on that list with `BRPOP`. Because `BRPOP` is atomic, N replicas can safely share one queue with no double-processing and no coordinator process.
4. A worker that pops a task ID re-reads the full task document from MongoDB, flips status to `running`, executes the requested string operation, and writes back `result` + `logs` + a terminal status (`success` / `failed`).
5. The frontend polls `GET /api/tasks/:id` every 2-4 seconds until the task reaches a terminal state, then stops polling. (A short-lived, sub-second-to-low-seconds job doesn’t justify a WebSocket/SSE channel; polling is simpler to reason about and to scale, and is swapped for push-based updates later only if job durations grow into minutes.)

Why the API and worker are separate processes, not just async middleware in Express: it lets each scale independently (the API scales on request volume, workers scale on queue depth), it isolates a worker crash from taking down user-facing auth/reads, and it keeps the “AI operation” logic in one place (Python) that’s easy to swap for real model inference later without touching the API.

Trust boundary: the frontend never talks to MongoDB or Redis directly — only the API does, and only the API and worker share the Mongo/Redis network. JWTs are validated on every authenticated route; task documents are always scoped to `req.user.id` so one user can never read or enumerate another’s tasks.

2. Worker Scaling Strategy

Workers are stateless consumers of a single Redis list, which makes horizontal scaling straightforward:

- **Kubernetes-native baseline:** a `Deployment` with `minReplicas: 2` and a CPU-based `HorizontalPodAutoscaler` (target 60% average utilization, up to 20 replicas). This is included in `infra/k8s/06-worker.yaml` and works out of the box on any cluster, k3s included.
- **Production recommendation — scale on queue depth, not CPU:** the string operations in this assignment are CPU-cheap, so CPU-based autoscaling under-reacts to a sudden backlog (the queue can grow to thousands of pending tasks while CPU stays low, because each pop-process-ack cycle is fast but the *arrival rate* is what’s spiking). The recommended fix is **KEDA** with a Redis trigger on `LLEN ai_task_queue`, scaling toward a target of ~20 pending tasks per worker replica. A commented-out `ScaledObject` for this is included alongside the HPA in `06-worker.yaml` so it’s a one-line uncomment once KEDA is installed in-cluster.
- **Graceful scale-down:** the worker’s signal handler lets an in-flight `BRPOP /task` finish before exiting on `SIGTERM`, so Kubernetes can scale the `Deployment` down without abandoning a task mid-processing (Kubernetes’ default 30s termination grace period is enough headroom for these short-lived operations; it would need increasing if operations grew longer-running).
- **Idempotency / retry safety:** if a worker pod is killed mid-task (e.g. node eviction) *before* it acknowledges completion, that task’s status stays `running` indefinitely rather than being requeued automatically — a deliberate choice to avoid double side effects from non-idempotent future operations (e.g. a paid external API call). A lightweight reconciliation job (a cron or a periodic check in the API) can sweep `running` tasks whose `startedAt` is older than a timeout and requeue them; this is out of scope for the current string operations but called out here because the “100k tasks/day” case below depends on it.

3. Handling High Task Volume (~100,000 tasks/day)

100,000 tasks/day averages to ~1.16 tasks/sec, which is trivial for this architecture at steady state — the real design problem is **bursts** (e.g. most of the day’s volume landing in a 1-2 hour window) and **write contention** on MongoDB, not raw throughput.

- **Queue as the shock absorber:** Redis absorbs burst arrival rate far above what workers can drain instantaneously; the API’s job is only to enqueue quickly (a single `LPUSH`) and return, so user-facing latency stays flat even while the queue backlog grows temporarily.
- **Horizontal worker scaling** (Section 2) drains that backlog by adding replicas proportional to queue depth rather than a fixed pool sized for worst case.
- **MongoDB write path:** at ~100k tasks/day, that’s ~100k inserts + ~200-300k updates/day (created → running → success/failed, each a targeted `$set / $push` on a single document by `_id`) — well within a single replica set’s capacity with the indexes in Section 4. If this were to grow another 1-2 orders of magnitude, the next steps would be read replicas for the dashboard’s list queries and, eventually, sharding on `user` (task documents are always accessed by user, making `user` a natural, well-distributed shard key).
- **Backpressure, not silent drops:** the API enforces per-IP rate limiting (`express-rate-limit`) so a single misbehaving client can’t monopolize the queue; task creation itself is never dropped for legitimate load, only slowed via the queue’s natural backlog.
- **Batching consideration:** if task volume grew enough that per-task Mongo round-trips became the bottleneck (not currently the case at 100k/day), the worker could batch status-update writes, but this assignment’s scale doesn’t warrant that complexity yet — premature batching would only obscure per-task logs and failure isolation.

4. MongoDB Indexing Strategy

Two access patterns dominate: (a) a user loading *their own* task dashboard, most recent first, optionally filtered by status, and (b) an operational query for stuck/pending tasks. `infra/the Task` model defines:

```
{ user: 1, createdAt: -1 } // dashboard: "my tasks, newest first"
{ user: 1, status: 1 } // dashboard: "my tasks, filtered by status"
{ status: 1, createdAt: 1 } // ops/reconciliation: "oldest pending tasks"
```

- The `{user, createdAt}` compound index directly covers the default dashboard query (`find({user}).sort({createdAt: -1})`) without a separate sort stage — this is the highest-traffic query in the system and the one most worth indexing precisely.
- `{user, status}` supports the “show me only failed/pending tasks” filter without falling back to a collection scan per user.

- `{status, createdAt}` is intentionally *not* prefixed by `user` — it exists for the reconciliation sweep described in Section 2 (“find pending tasks older than N minutes across all users”), which needs to scan by status globally, not per-user.
- `email` on the `User` collection is a unique index (declared via the schema’s `unique: true`), which both enforces the “no duplicate accounts” constraint at the database level and makes login lookups $O(\log n)$.
- Deliberately **not** indexed: `inputText` and `result` (free text, never queried/filtered on) and `logs` (an array only ever read wholesale per-document, never searched). Indexing them would cost write throughput on every task update for no query benefit.

5. Redis Failure Handling and Recovery Strategy

Redis plays two roles here — the task queue and (implicitly) a single point of coordination between API and workers — so its failure modes need explicit handling on both sides:

- **API side:** the Redis client (`redis` npm package) is configured with a reconnect strategy (capped exponential backoff, `config/redis.js`). If `LPUSH` fails because Redis is unreachable, task *creation* fails loudly (the API returns an error to the user) rather than silently accepting a task that will never be processed — the Mongo write and the queue push are not wrapped in a distributed transaction, so we choose to fail closed on the enqueue step specifically to avoid orphaned `pending` tasks that look accepted but never run.
- **Worker side:** `main.py` retries the Redis connection in a loop with a fixed backoff before the worker even starts consuming, and on any `RedisError` during the main loop it reconnects and continues rather than crashing the pod (which would otherwise trigger a crash-loop-backoff cycle in Kubernetes for a transient blip).
- **Data durability:** Redis is run with `appendonly yes` (AOF persistence, see `docker-compose.yml` and `infra/k8s/04-redis.yml`), so a Redis pod restart replays the AOF log and does not silently drop queued task IDs that hadn’t been popped yet. AOF is the right tradeoff here over RDB snapshotting, since losing even a few seconds of unprocessed task IDs on a crash is a real (if rare) user-visible bug.
- **Production hardening beyond this assignment’s scope:** for a real production deployment, single-instance Redis is a single point of failure regardless of AOF (the *node* itself can die). The recommended upgrade path is either a managed Redis (e.g. Elasticache/Upstash) with automatic failover, or Redis Sentinel/Cluster in-cluster. Because task IDs are cheap to regenerate from MongoDB, an additional safety net is the reconciliation sweep from Section 2: even in a worst-case scenario where the Redis queue is lost entirely, any task still `pending` in MongoDB can be re-enqueued by a recovery script, making MongoDB (not Redis) the source of truth for “did this task get created,” and Redis purely a delivery mechanism.

6. Deployment Strategy: Staging vs. Production

Both environments deploy from the same base Kubernetes manifests (`infra/k8s/`), diverging only through Kustomize overlays (`infra/overlays/staging` , `infra/overlays/production`) — this keeps the two environments structurally identical (same probes, same resource shape, same Services/Ingress paths) and prevents “works in staging, breaks in production” drift from manifest divergence.

	Staging	Production
Namespace	<code>ai-task-platform-staging</code>	<code>ai-task-platform</code>
Replicas	1 per component (cost-optimized)	2+ per component, HPA-scaled
Image tag	<code>:staging</code> (updated on every merge to <code>main</code>)	immutable commit-SHA tag, promoted deliberately
Ingress host	<code>staging.ai-task-platform.example.com</code>	<code>ai-task-platform.example.com</code>
Argo CD Application	<code>ai-task-platform-staging</code>	<code>ai-task-platform-production</code>

Promotion flow: CI builds and pushes images tagged with the commit SHA on every merge to `main` (`.github/workflows/ci-cd.yml`), then auto-updates the *staging* overlay’s image tag as part of the same pipeline run. Staging is therefore always tracking `main` directly via Argo CD’s auto-sync — it is the continuous integration target, not a manually-gated environment. Promotion to *production* is a deliberate, reviewed step: a maintainer opens a PR against the Infrastructure Repository bumping `overlays/production/kustomization.yml` ’s image tags to a specific, already-staging-verified commit SHA. Merging that PR is what triggers Argo CD to roll production forward — this gives production a human approval gate (the PR review) while staging stays fully automatic, without needing two different CI pipelines.

Why Argo CD auto-sync is safe here: `syncPolicy.automated.selfHeal: true` means any manual `kubectl edit` against the live cluster gets reverted back to match git within moments — this is intentional. The Infrastructure Repository, not `kubectl` , is the single source of truth for both environments, which is the core GitOps guarantee this assignment asks for.

Rollback: because promotion to production is a git commit, rollback is `git revert` on that commit (or re-running the PR merge with the previous SHA) — Argo CD picks up the reverted state and rolls the cluster back automatically, with no manual `kubectl rollout undo` step needed.